# Local search report


## Paweł Charkiewicz 160288, Piotr Franc 160306


Objective = cycle length (Euclidean, rounded to nearest integer) + sum of selected node costs. Lower is better.

## Pseudocode

All methods read the instance into masterArray, which stores every node (x, y, cost). M = number of runs = 200, N = solution size = ⌈n/2⌉, d(a, b) = Euclidean distance rounded to the nearest integer, s = the current solution (a cycle s[0] → s[1] → ... → s[N − 1] → s[0]), prev(i) / next(i) = the nodes before / after position i on the cycle, Δ = change of the objective (negative = improvement). Every method is one option from each of three groups: the starting solution (InitialSolutionsType), the intra-route move (NeighborhoodType) and the type of local search (SearchMode). The inter-route move is always the exchange of a selected and a not selected node. All Δ are computed from the changed edges only; no neighbour solution is built to evaluate a move.

### Local search (common loop)

```
0. Compute an n × n distance matrix d between every pair of nodes.
1. Build the move lists once:
       exchanges:  every pair (position i in s, index j in the list of not selected nodes)
       node swaps: every pair of positions (i, j), i < j
       edge swaps: every pair of positions (i, j), j ≥ i + 2, except (0, N − 1)
                   (the two edges would share node s[0])
2. Create M starting solutions (RANDOM or GREEDY_REGRET), timing the construction of each one.
3. For every starting solution s:
4.     Start a second timer for the search.
5.     Repeat:
6.         unused = nodes not in s.
7.         Shuffle the exchange list and the intra-route move list.
8.         Run one step of the search (GREEDY or STEEP); it applies at most one move.
9.     Until the step finds no move with Δ < 0.
10.    Stop the timer and save s and its running time = construction time + search time.
```

### Inter-route move: exchange of a selected and a not selected node

```
1. Take position i (node v = s[i], neighbours a = prev(i), b = next(i)) and a not selected node u.
2. Δ = d(a, u) + d(u, b) + cost(u) − d(a, v) − d(v, b) − cost(v)
3. Apply: put u at position i, mark v as not selected and u as selected.
```

### NODE_SWAP: two-nodes exchange (intra-route)

```
1. Take positions i < j, nodes x = s[i] and y = s[j].
2. If y comes right after x (… a, x, y, b …):
       Δ = d(a, y) + d(x, b) − d(a, x) − d(y, b)
3. Else if x comes right after y (i = 0, j = N − 1, the cycle wraps: … a, y, x, b …):
       Δ = d(a, x) + d(y, b) − d(a, y) − d(x, b)
4. Otherwise (… a, x, b … c, y, e …):
       Δ = d(a, y) + d(y, b) + d(c, x) + d(x, e) − d(a, x) − d(x, b) − d(c, y) − d(y, e)
5. Apply: swap s[i] and s[j].
```

### EDGE_SWAP: two-edges exchange (intra-route)

```
1. Take positions i < j (from the edge swap list), edges (s[i], s[i+1]) and (s[j], s[j+1]).
2. Δ = d(s[i], s[j]) + d(s[i+1], s[j+1]) − d(s[i], s[i+1]) − d(s[j], s[j+1])
3. Apply: reverse the segment s[i+1] … s[j]; the cycle now uses edges (s[i], s[j]) and (s[i+1], s[j+1]).
```

### STEEP: steepest local search (one step)

```
1. For every exchange in the list: compute Δ, remember the smallest one (bestInter).
2. For every intra-route move in the list: compute Δ, remember the smallest one (bestIntra).
3. If bestInter ≥ 0 and bestIntra ≥ 0: s is a local optimum, stop.
4. If bestInter < bestIntra: apply the best exchange.
5. Otherwise apply the best intra-route move.
```

### GREEDY: greedy local search (one step)

```
1. Both move lists were just shuffled. Set pointers e = 0 (exchanges) and t = 0 (intra-route moves).
2. While some move has not been evaluated:
3.     If both lists still have moves: toss a fair coin to pick the list.
       Otherwise: pick the list that still has moves.
4.     Take the next move from the picked list and compute its Δ.
5.     If Δ < 0: apply the move and end the step (improvement found).
6. If every move was evaluated without improvement: s is a local optimum, stop.
```

### RANDOM: random starting solution

```
1. Start with an empty cycle.
2. Until it holds N nodes: draw a random node index that is not yet used and append it.
```

### GREEDY_REGRET: greedy cycle + weighted sum starting solution

```
1. Run m (m = 0 … 199) starts from node m alone.
2. While the cycle holds fewer than N nodes:
3.     For every unused node k, compute the insertion cost for every edge (a, b) of the cycle,
       including the closing edge:
           d(a, k) + d(k, b) − d(a, b) + cost(k)
       Take the smallest value best1(k), the second smallest best2(k), and the edge pos(k)
       that gives best1(k). regret(k) = best2(k) − best1(k) (0 when there is only one edge).
4.     score(k) = w1 · regret(k) − w2 · best1(k),    w1 = 1, w2 = 1.5
5.     Choose the node k with the greatest score (ties: smaller best1), insert it into the edge pos(k)
       and mark it as used.
```

## Randomisation in the greedy version

Before every step both move lists (the exchanges and the intra-route moves) are shuffled. The greedy search then walks through them with two pointers: while both lists still have moves, a fair coin decides whether the next evaluated move is an exchange or an intra-route move; when one list is exhausted the other one is continued. The first move with $\Delta < 0$ is applied, so only the moves up to that one are evaluated. Neither kind of move is always tried first.

# Computational experiment

Instances: TSPA, TSPB. Eight methods (steepest / greedy × two-nodes / two-edges exchange × random / greedy start), each run 200 times. Each cell: average (min – max) over the valid runs; the best average on each instance is in bold.

## Objective function

| Method | TSPA | TSPB |
|---|---:|---:|
| Steepest LS, two-nodes exchange, random start | 90595.49 (84540 – 103667) | 65972.00 (59633 – 73790) |
| Steepest LS, two-nodes exchange, greedy start | 72908.71 (71955 – 75030) | 55013.50 (52169 – 61238) |
| Steepest LS, two-edges exchange, random start | 76116.04 (73733 – 79516) | **52552.71 (50051 – 55367)** |
| Steepest LS, two-edges exchange, greedy start | **72881.93 (71955 – 74666)** | 54820.22 (52034 – 60749) |
| Greedy LS, two-nodes exchange, random start | 87921.74 (80782 – 96011) | 64030.60 (58188 – 69686) |
| Greedy LS, two-nodes exchange, greedy start | 72916.29 (71955 – 75030) | 55015.43 (52169 – 61238) |
| Greedy LS, two-edges exchange, random start | 76414.85 (74309 – 79765) | 52920.14 (50760 – 55422) |
| Greedy LS, two-edges exchange, greedy start | 72892.15 (71955 – 74859) | 54816.94 (52034 – 60749) |

## Running time [ms]

Time of one run: building its starting solution + the local search.

| Method | TSPA | TSPB |
|---|---:|---:|
| Steepest LS, two-nodes exchange, random start | 341.37 (262 – 480) | 336.37 (271 – 441) |
| Steepest LS, two-nodes exchange, greedy start | 53.56 (42 – 77) | 63.58 (45 – 93) |
| Steepest LS, two-edges exchange, random start | 235.13 (198 – 272) | 236.94 (198 – 286) |
| Steepest LS, two-edges exchange, greedy start | 50.87 (42 – 72) | 62.03 (44 – 90) |
| Greedy LS, two-nodes exchange, random start | 62.84 (45 – 407) | 64.17 (45 – 88) |
| Greedy LS, two-nodes exchange, greedy start | 44.75 (40 – 69) | 51.99 (42 – 69) |
| Greedy LS, two-edges exchange, random start | 50.25 (40 – 70) | 54.60 (44 – 79) |
| Greedy LS, two-edges exchange, greedy start | **43.15 (40 – 62)** | **50.49 (41 – 68)** |

## Validation

Valid = the run has ⌈n/2⌉ distinct nodes of the instance. Local optima = no improving move remains in the method's neighbourhood (checked independently by the report script; "best Δ" is the most improving move left).

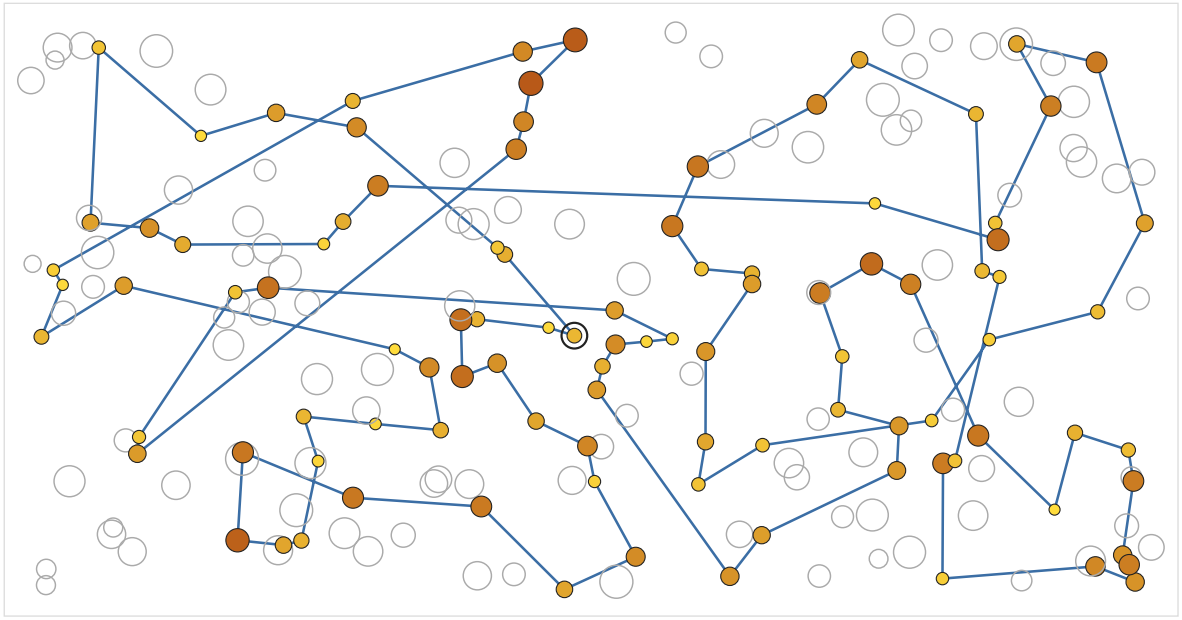
| Method | TSPA valid | TSPA local optima | TSPB valid | TSPB local optima |
|---|---|---|---|---|
| Steepest LS, two-nodes exchange, random start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Steepest LS, two-nodes exchange, greedy start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Steepest LS, two-edges exchange, random start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Steepest LS, two-edges exchange, greedy start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Greedy LS, two-nodes exchange, random start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Greedy LS, two-nodes exchange, greedy start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Greedy LS, two-edges exchange, random start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |
| Greedy LS, two-edges exchange, greedy start | **200 / 200** | **200 / 200** | **200 / 200** | **200 / 200** |

**All runs valid and all runs are local optima.**

# Instance TSPA

## Steepest LS, two-nodes exchange, random start

Best of 200 valid runs: run #7 with objective **84540**.



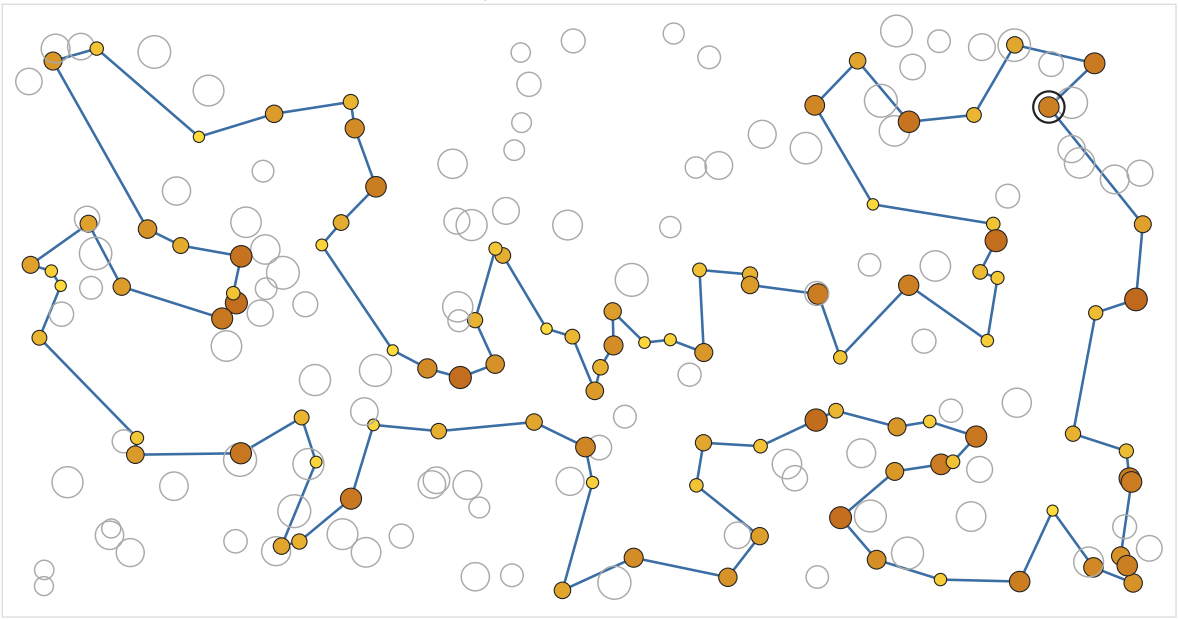
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
144, 67, 35, 156, 102, 185, 45, 73, 135, 181, 114, 98, 189, 161, 92, 171, 2, 187, 30, 141, 87, 28, 180, 49, 168,
63, 149, 157, 115, 175, 5, 112, 139, 127, 97, 123, 90, 52, 36, 177, 179, 53, 140, 192, 158, 151, 150, 176, 174,
8, 142, 85, 113, 107, 130, 159, 170, 153, 193, 78, 88, 57, 42, 89, 50, 3, 178, 136, 186, 62, 143, 33, 9, 54,
131, 84, 58, 134, 106, 93, 47, 147, 55, 29, 101, 1, 169, 164, 4, 76, 132, 152, 108, 60, 48, 80, 91, 71, 51, 16
```

## Steepest LS, two-nodes exchange, greedy start

Best of 200 valid runs: run #6 with objective **71955**.



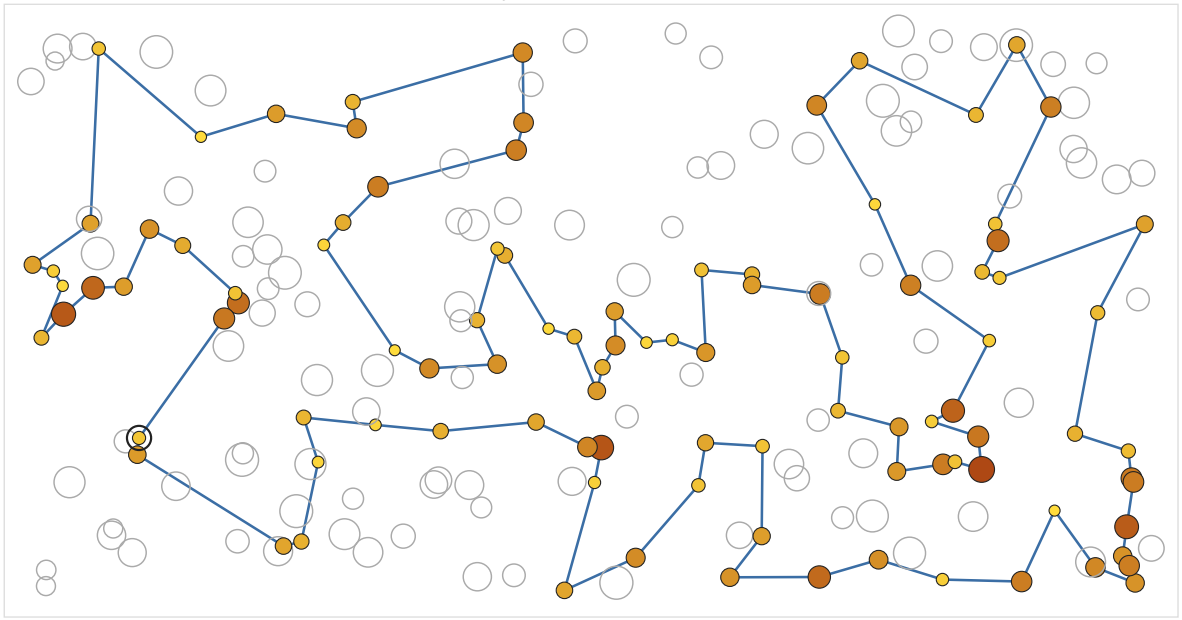
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
101, 55, 29, 178, 184, 136, 186, 164, 1, 169, 3, 50, 93, 85, 174, 8, 54, 9, 33, 131, 36, 177, 52, 179, 53, 140,
144, 67, 16, 51, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48, 31, 108, 152, 100, 123, 83, 160, 49,
60, 116, 149, 63, 168, 97, 127, 161, 30, 187, 171, 2, 189, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84,
134, 72, 176, 150, 106, 113, 89, 42, 151, 138, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 47, 37,
147
```

## Steepest LS, two-edges exchange, random start

Best of 200 valid runs: run #46 with objective **73733**.



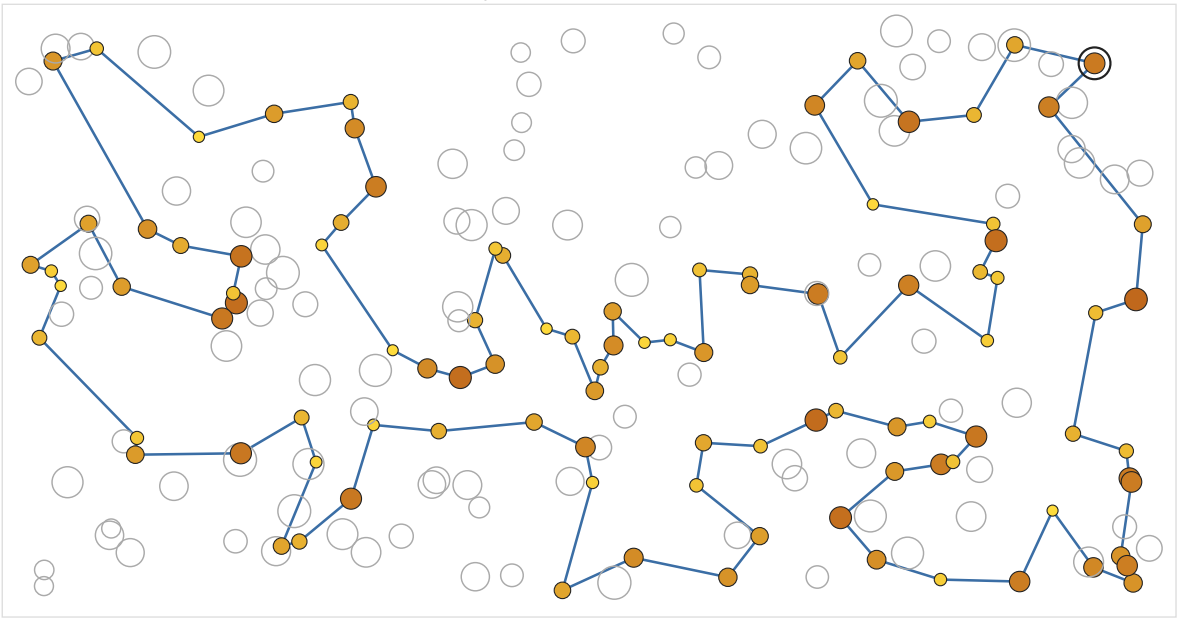
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
97, 160, 83, 123, 152, 108, 49, 11, 119, 168, 63, 149, 116, 60, 48, 80, 91, 71, 157, 115, 112, 139, 4, 76, 132,
180, 28, 185, 35, 51, 16, 67, 144, 140, 53, 179, 52, 177, 36, 131, 33, 9, 54, 8, 174, 176, 150, 151, 42, 89,
103, 113, 106, 43, 93, 85, 164, 186, 136, 178, 29, 101, 1, 169, 3, 50, 147, 47, 130, 159, 146, 170, 75, 153,
193, 78, 88, 107, 64, 57, 182, 111, 192, 158, 134, 84, 58, 181, 114, 135, 61, 73, 45, 87, 141, 30, 187, 2, 171,
127
```

## Steepest LS, two-edges exchange, greedy start

Best of 200 valid runs: run #6 with objective **71955**.



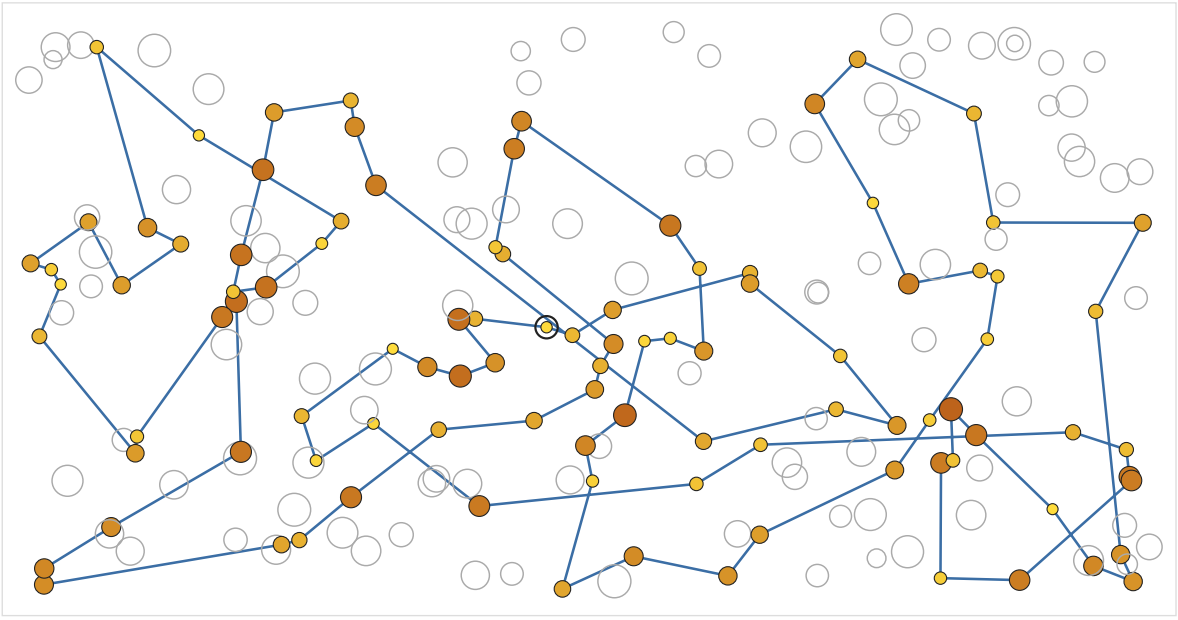
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
55, 101, 147, 37, 47, 130, 159, 146, 170, 153, 193, 78, 88, 107, 64, 57, 182, 138, 151, 42, 89, 113, 106, 150,
176, 72, 134, 84, 58, 158, 192, 181, 114, 135, 73, 45, 87, 141, 189, 2, 171, 187, 30, 161, 127, 97, 168, 63,
149, 116, 60, 49, 160, 83, 123, 100, 152, 108, 31, 48, 80, 91, 157, 71, 4, 76, 132, 180, 28, 102, 185, 35, 51,
16, 67, 144, 140, 53, 179, 52, 177, 36, 131, 33, 9, 54, 8, 174, 85, 93, 50, 3, 169, 1, 164, 186, 136, 184, 178,
29
```

## Greedy LS, two-nodes exchange, random start

Best of 200 valid runs: run #199 with objective **80782**.



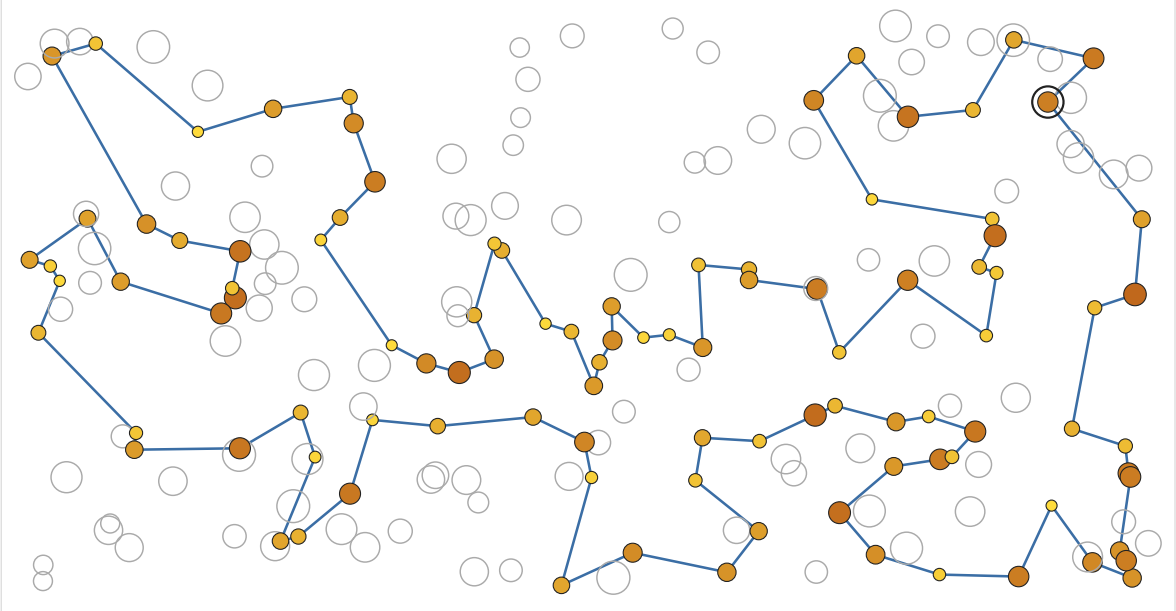
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
67, 35, 156, 185, 102, 28, 180, 30, 187, 141, 98, 58, 134, 130, 159, 146, 170, 64, 57, 42, 89, 43, 113, 107, 88,
78, 153, 47, 147, 1, 178, 136, 186, 164, 85, 3, 50, 93, 106, 151, 158, 192, 181, 114, 135, 73, 41, 177, 36, 131,
33, 143, 112, 139, 51, 16, 179, 53, 140, 45, 87, 189, 2, 171, 104, 129, 69, 161, 83, 160, 97, 127, 168, 63, 149,
116, 60, 49, 152, 108, 48, 80, 76, 132, 90, 123, 100, 38, 91, 157, 71, 4, 84, 176, 150, 174, 54, 9, 52, 144
```

## Greedy LS, two-nodes exchange, greedy start

Best of 200 valid runs: run #6 with objective **71955**.



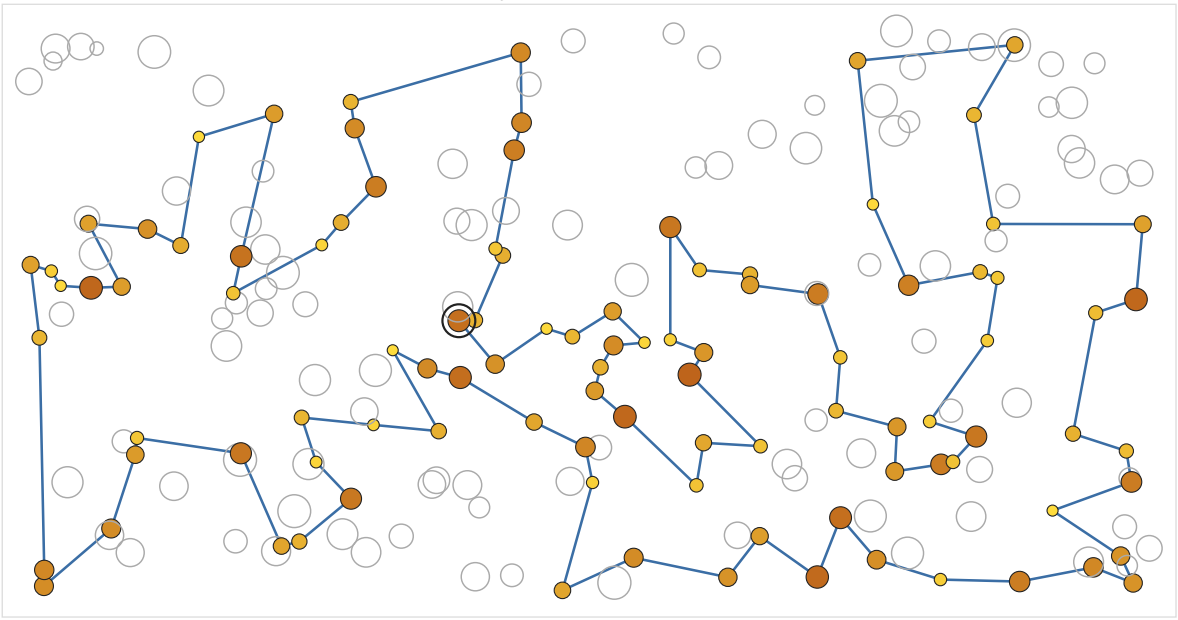
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
101, 55, 29, 178, 184, 136, 186, 164, 1, 169, 3, 50, 93, 85, 174, 8, 54, 9, 33, 131, 36, 177, 52, 179, 53, 140,
144, 67, 16, 51, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48, 31, 108, 152, 100, 123, 83, 160, 49,
60, 116, 149, 63, 168, 97, 127, 161, 30, 187, 171, 2, 189, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84,
134, 72, 176, 150, 106, 113, 89, 42, 151, 138, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 47, 37,
147
```

## Greedy LS, two-edges exchange, random start

Best of 200 valid runs: run #90 with objective **74309**.



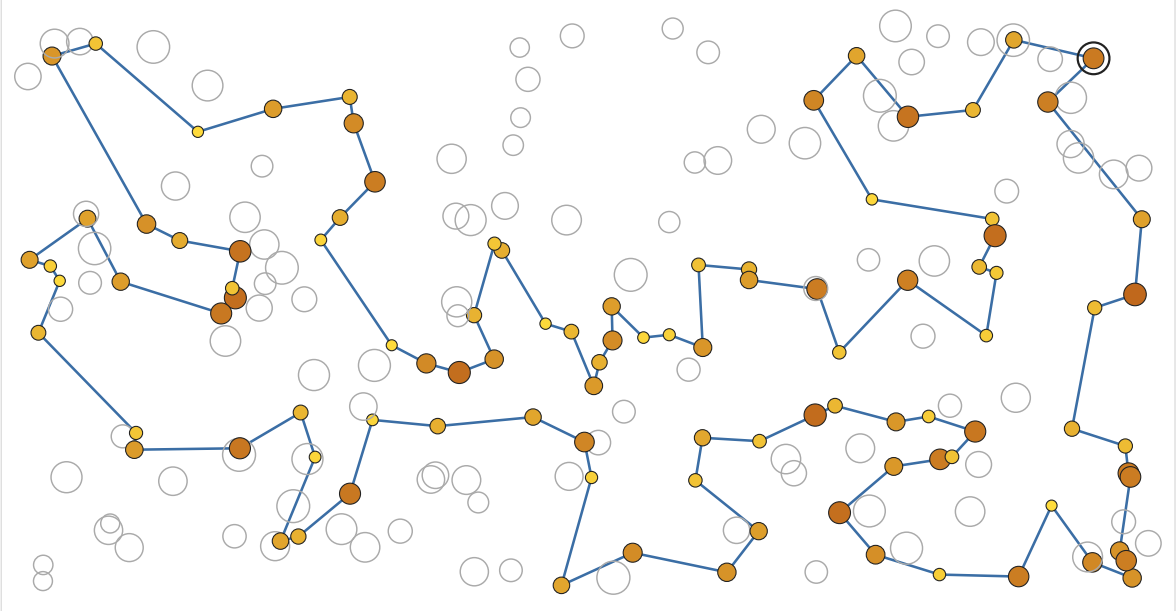
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
156, 185, 67, 144, 52, 177, 179, 53, 140, 41, 58, 84, 134, 99, 131, 36, 143, 33, 9, 54, 8, 174, 176, 150, 151,
42, 89, 113, 106, 93, 50, 3, 85, 164, 136, 29, 178, 1, 147, 37, 47, 130, 159, 170, 107, 153, 78, 88, 64, 57,
182, 138, 111, 158, 192, 181, 114, 135, 73, 45, 102, 28, 180, 87, 141, 30, 187, 189, 2, 171, 161, 97, 127, 69,
104, 129, 168, 116, 149, 63, 11, 49, 60, 108, 152, 80, 91, 100, 123, 132, 76, 4, 71, 157, 115, 112, 139, 51, 16,
35
```

## Greedy LS, two-edges exchange, greedy start

Best of 200 valid runs: run #6 with objective **71955**.



node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

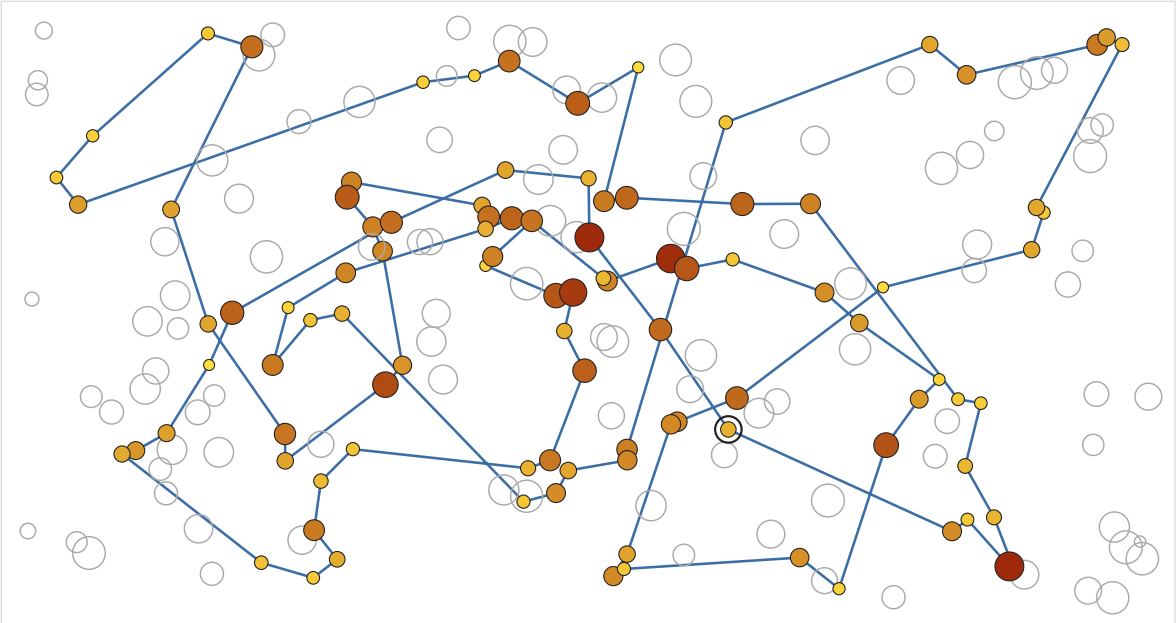
### Best solution (node indices, from 0)

```
55, 101, 147, 37, 47, 130, 159, 146, 170, 153, 193, 78, 88, 107, 64, 57, 182, 138, 151, 42, 89, 113, 106, 150,
176, 72, 134, 84, 58, 158, 192, 181, 114, 135, 73, 45, 87, 141, 189, 2, 171, 187, 30, 161, 127, 97, 168, 63,
149, 116, 60, 49, 160, 83, 123, 100, 152, 108, 31, 48, 80, 91, 157, 71, 4, 76, 132, 180, 28, 102, 185, 35, 51,
16, 67, 144, 140, 53, 179, 52, 177, 36, 131, 33, 9, 54, 8, 174, 85, 93, 50, 3, 169, 1, 164, 186, 136, 184, 178,
29
```

# Instance TSPB

## Steepest LS, two-nodes exchange, random start

Best of 200 valid runs: run #20 with objective **59633**.



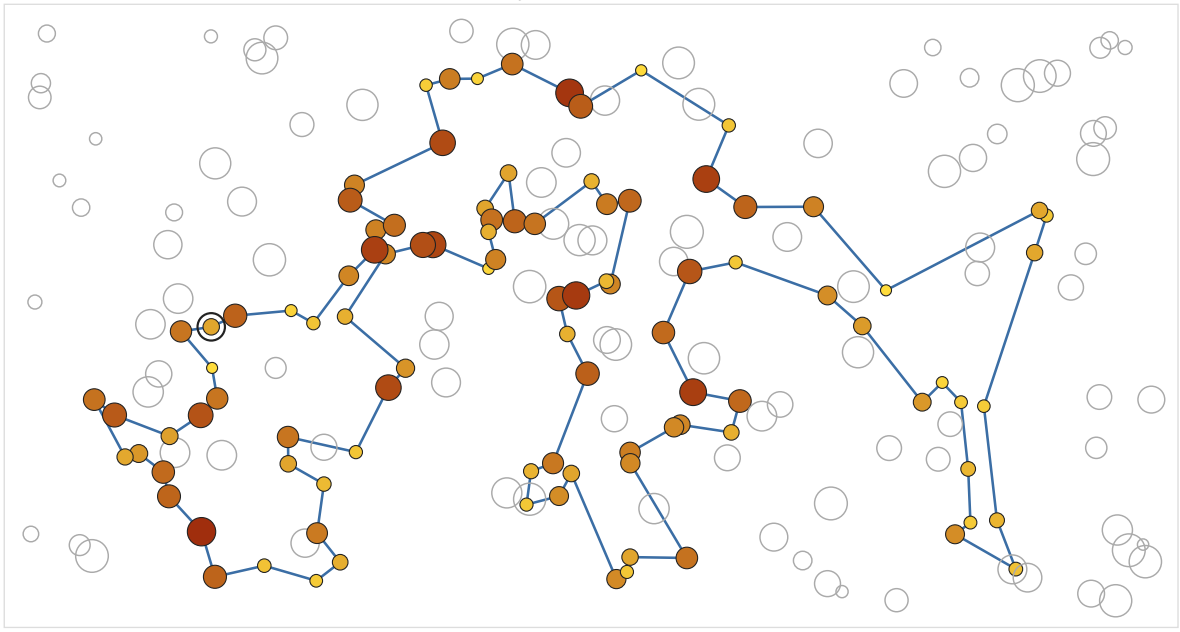
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
67, 74, 20, 188, 2, 14, 153, 90, 129, 104, 185, 86, 46, 89, 156, 48, 96, 121, 145, 28, 149, 116, 147, 158, 125,
155, 71, 33, 64, 21, 30, 191, 130, 133, 136, 8, 172, 38, 154, 97, 140, 47, 85, 51, 167, 181, 152, 57, 190, 70,
81, 94, 105, 196, 114, 10, 54, 66, 24, 34, 179, 82, 87, 99, 171, 150, 80, 127, 192, 162, 76, 135, 75, 3, 184,
117, 160, 69, 122, 173, 151, 98, 73, 59, 40, 60, 157, 45, 134, 175, 132, 29, 6, 170, 26, 183, 120, 72, 137, 27
```

## Steepest LS, two-nodes exchange, greedy start

Best of 200 valid runs: run #125 with objective **52169**.



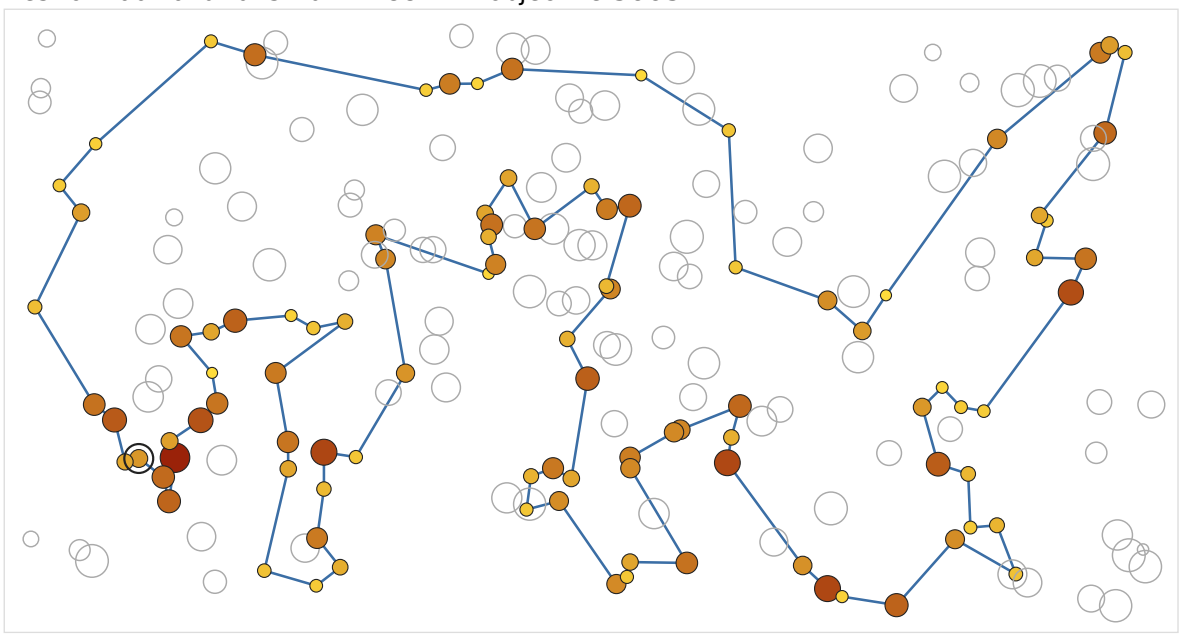
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
125, 26, 154, 140, 38, 176, 30, 183, 191, 130, 63, 121, 198, 96, 48, 43, 156, 89, 57, 79, 185, 104, 54, 196,
114, 10, 90, 14, 2, 74, 20, 153, 129, 127, 80, 192, 162, 76, 135, 27, 11, 66, 67, 24, 34, 152, 181, 88, 179, 87,
82, 167, 51, 85, 59, 73, 98, 151, 122, 173, 184, 3, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 168, 189,
21, 47, 64, 33, 40, 155, 71, 60, 157, 45, 134, 175, 62, 91, 36, 197, 29, 132, 178, 103, 6, 180, 115, 170, 177
```

## Steepest LS, two-edges exchange, random start

Best of 200 valid runs: run #188 with objective **50051**.



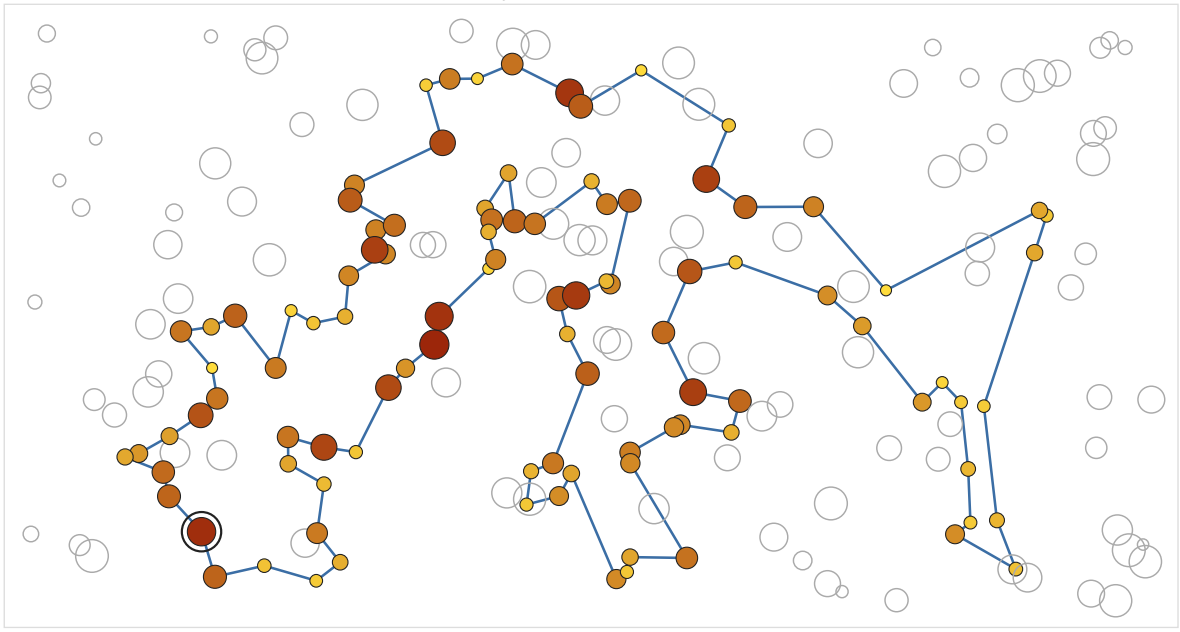
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
29, 132, 103, 178, 65, 145, 28, 149, 116, 147, 121, 198, 96, 48, 89, 57, 76, 162, 192, 54, 187, 81, 94, 105, 61,
196, 114, 10, 148, 143, 90, 129, 127, 80, 182, 153, 20, 14, 2, 74, 17, 171, 39, 99, 31, 67, 66, 24, 34, 152,
181, 88, 179, 87, 82, 51, 85, 59, 73, 167, 98, 151, 3, 184, 86, 46, 72, 117, 120, 133, 136, 172, 160, 69, 30,
21, 64, 40, 141, 60, 157, 45, 134, 175, 71, 155, 97, 47, 140, 154, 26, 125, 177, 170, 115, 180, 6, 5, 36, 197
```

## Steepest LS, two-edges exchange, greedy start

Best of 200 valid runs: run #91 with objective **52034**.



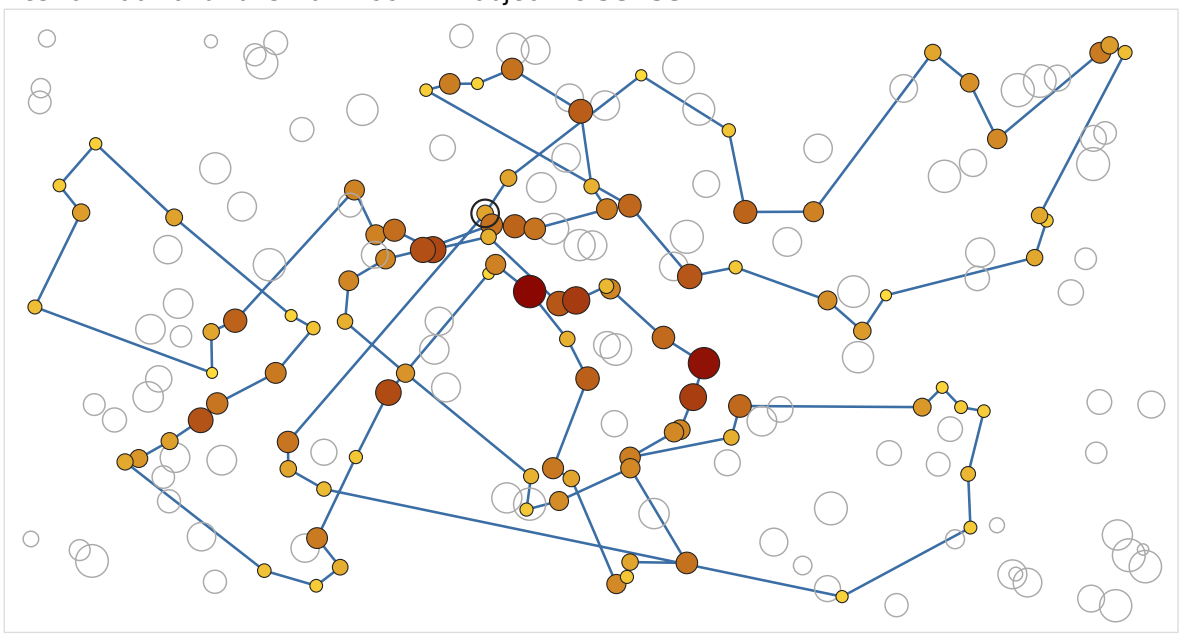
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
91, 36, 197, 132, 29, 6, 180, 115, 170, 177, 125, 26, 97, 154, 140, 47, 38, 21, 176, 30, 183, 191, 130, 63, 121,
198, 96, 48, 43, 156, 89, 57, 79, 185, 104, 54, 196, 114, 10, 90, 14, 2, 74, 20, 153, 129, 127, 80, 192, 162,
76, 135, 27, 11, 66, 67, 24, 34, 152, 181, 88, 179, 87, 82, 167, 51, 85, 59, 73, 98, 151, 122, 173, 184, 3, 86,
46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 124, 4, 64, 33, 40, 141, 155, 71, 60, 157, 45, 134, 175, 62
```

## Greedy LS, two-nodes exchange, random start

Best of 200 valid runs: run #99 with objective **58188**.



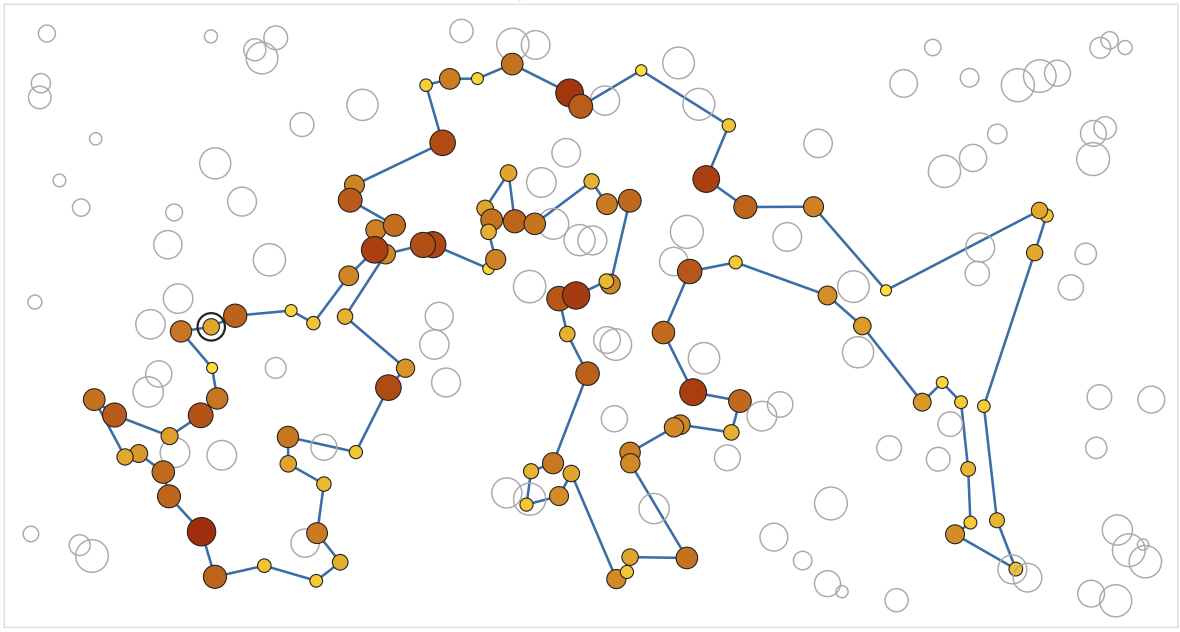
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
133, 120, 89, 57, 185, 104, 190, 70, 187, 81, 94, 105, 196, 114, 10, 54, 192, 162, 76, 135, 86, 121, 198, 96,
48, 156, 72, 46, 117, 8, 136, 189, 21, 38, 47, 64, 59, 85, 51, 181, 88, 179, 87, 82, 167, 73, 98, 151, 128, 160,
69, 33, 40, 157, 45, 134, 175, 132, 29, 6, 180, 115, 97, 140, 154, 158, 149, 28, 145, 65, 170, 125, 26, 130, 30,
183, 168, 172, 122, 173, 184, 3, 27, 35, 11, 24, 34, 152, 67, 66, 80, 127, 129, 90, 153, 20, 171, 60, 71, 155
```

## Greedy LS, two-nodes exchange, greedy start

Best of 200 valid runs: run #125 with objective **52169**.



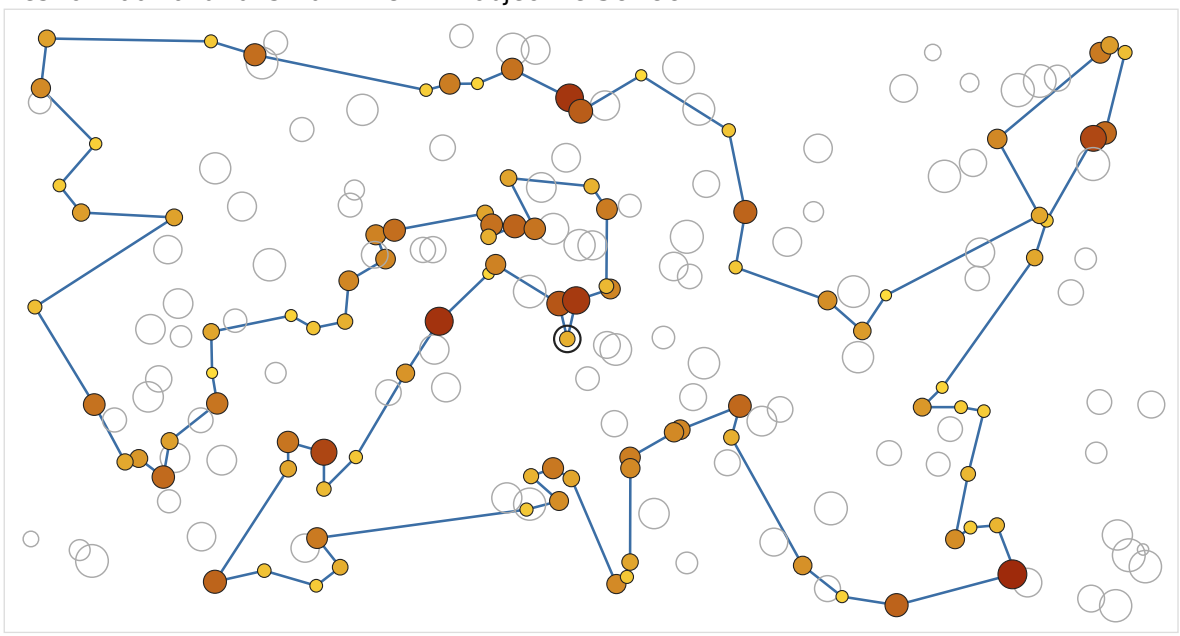
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
125, 26, 154, 140, 38, 176, 30, 183, 191, 130, 63, 121, 198, 96, 48, 43, 156, 89, 57, 79, 185, 104, 54, 196,
114, 10, 90, 14, 2, 74, 20, 153, 129, 127, 80, 192, 162, 76, 135, 27, 11, 66, 67, 24, 34, 152, 181, 88, 179, 87,
82, 167, 51, 85, 59, 73, 98, 151, 122, 173, 184, 3, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 168, 189,
21, 47, 64, 33, 40, 155, 71, 60, 157, 45, 134, 175, 62, 91, 36, 197, 29, 132, 178, 103, 6, 180, 115, 170, 177
```

## Greedy LS, two-edges exchange, random start

Best of 200 valid runs: run #18 with objective **50760**.



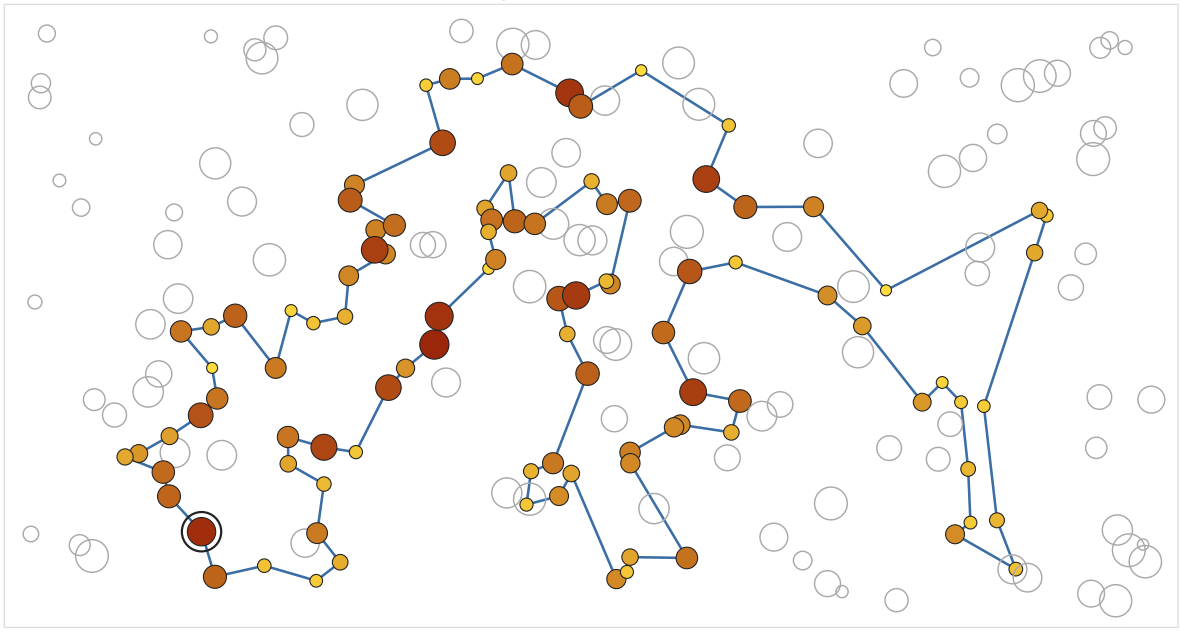
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

### Best solution (node indices, from 0)

```
151, 122, 160, 69, 124, 64, 40, 60, 141, 155, 71, 62, 175, 134, 45, 157, 85, 51, 59, 73, 167, 82, 87, 179, 181,
152, 34, 24, 66, 67, 99, 171, 17, 188, 2, 14, 20, 74, 153, 90, 129, 80, 127, 10, 114, 139, 61, 105, 94, 81, 187,
196, 54, 192, 162, 76, 185, 57, 89, 156, 43, 48, 96, 198, 121, 147, 116, 37, 146, 149, 28, 145, 158, 65, 178,
132, 29, 197, 6, 115, 170, 125, 154, 140, 47, 38, 21, 30, 183, 133, 136, 172, 8, 117, 120, 72, 46, 184, 3, 173
```

### Greedy LS, two-edges exchange, greedy start

Best of 200 valid runs: run #91 with objective **52034**.



node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

**Best solution (node indices, from 0)**

```
91, 36, 197, 132, 29, 6, 180, 115, 170, 177, 125, 26, 97, 154, 140, 47, 38, 21, 176, 30, 183, 191, 130, 63, 121,
198, 96, 48, 43, 156, 89, 57, 79, 185, 104, 54, 196, 114, 10, 90, 14, 2, 74, 20, 153, 129, 127, 80, 192, 162,
76, 135, 27, 11, 66, 67, 24, 34, 152, 181, 88, 179, 87, 82, 167, 51, 85, 59, 73, 98, 151, 122, 173, 184, 3, 86,
46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 124, 4, 64, 33, 40, 141, 155, 71, 60, 157, 45, 134, 175, 62
```

Source code: https://github.com/CodeSencor/evolutionary-computation

## Conclusions

Steepest search beats the greedy search in quality at the cost of performance. Our longest runs were all in steepest mode, which resulted in more than four times the expected cost per run. Greedy + 2-regret provides a very strong baseline solution which does not leave much room for improvement even in steepest mode, resulting in shorter runs. Quite unexpectedly the best performance overall was achieved by a random sourced solution from TSPB dataset. It can be hypothesized that stochasticity in this scenario is beneficial, as it provides explorativeness to environment where exploitativeness is limited.